

PROJECT REPORT

Pearls AQI Predictor

End-to-End Air Quality Forecasting and MLOps System

Shiza Aamir

9/6/2026

Table of Contents

Contents

1. Executive Summary	5
2. Project Objective and Requirements	6
3. Technology Selection and Free Alternatives	7
4. Data Collection and Historical Backfilling	9
5. AQI Target Construction	11
6. Exploratory Data Analysis	11
7. Feature Engineering	13
8. Feature Store Implementation	15
9. Train/Validation/Test Strategy	15
10. Models Experimented With	16
11. Why XGBoost Was Selected	17
12. Final Holdout Evaluation	18
13. Explainability	19
14. Model Registry and MLOps	20
15. Backend Development	21
16. Frontend Dashboard	22
17. Automation with GitHub Actions	25
18. Production Monitoring	26
19. Serverless Persistence Challenge	26
20. Other Problems Faced and Their Resolution	27
20.1 Historical API Limitations	27
20.2 Hopsworks Cluster Provisioning	27
20.3 Large Dataset Size	27
20.4 Time-Series Leakage Risk	28
20.5 Deep Models Were More Expensive Without Clear Benefit	28
20.6 GitHub Action Parquet Error	28
20.7 External API Timeout	28
20.8 History Endpoint Dependency	28
20.9 Vercel Ephemeral Storage	29
20.10 Free Hosting Constraints	29
20.11 Vercel SPA Configuration Failure	29

21. Testing and Validation	29
22. Final Deployment	30
23. Final System Architecture.....	30
24. Results and Achievements.....	31
25. Limitations.....	32
26. Future Improvements	32
27. Conclusion	33

Table of Tables

Table 1: Final holdout performance of the selected XGBoost forecasting models.	5
Table 2 Comparison of originally suggested technologies and the free alternatives adopted during implementation.	7
Table 3 Final temporal training and holdout dataset configuration.	16
Table 4 Forecasting models evaluated during model experimentation and benchmarking.	16
Table 5 Mean walk-forward validation MAE of the classical forecasting models across all prediction horizons.	17
Table 6 Mean walk-forward validation performance of the selected XGBoost models.	18
Table 7 Final XGBoost holdout performance compared with the persistence baseline.	18
Table 8 Summary of the completed Pearls AQI Predictor system	31

Table of Figures

Figure 1 Pearls AQI Predictor production dashboard displaying current air quality and 24-, 48-, and 72-hour AQI forecasts.	6
Figure 2 Feast feature store configuration and successful feature repository validation.....	9
Figure 3 Historical environmental-data backfill summary prior to final ML dataset filtering.	10
Figure 4 Sample of the final ML-ready dataset containing meteorological variables, pollutant measurements, AQI values, and multi-horizon forecasting targets.....	10
Figure 5: Exploratory analysis of AQI distribution and city-level average air quality across the final dataset.....	12
Figure 6: Average historical AQI across the nine Pakistani cities included in the final modeling dataset.	12
Figure 7: Monthly mean AQI illustrating seasonal variation in historical air-quality conditions.	12
Figure 8 Correlation matrix showing relationships among AQI, particulate pollutants, and major meteorological variables.....	13
Figure 9 Summary and quality validation of the final ML-ready dataset used for model development.....	14
Figure 10 Temporal walk-forward validation strategy and isolated final holdout period used to prevent time-series data leakage.....	16
Figure 11: Comparison of candidate forecasting models using mean walk-forward validation MAE across the three forecast horizons.	17
Figure 12 Performance of the selected XGBoost models on the frozen final holdout dataset.	19
Figure 13: XGBoost feature-contribution explanation showing the variables with the strongest influence on an individual AQI forecast.....	20
Figure 14: MLflow model-management interface used to track and release the production XGBoost forecasting models.....	21
Figure 15: Swagger/OpenAPI documentation for the deployed FastAPI prediction service.	22
Figure 16: Overview page of the production dashboard displaying live AQI conditions, alerts, and multi-horizon forecasts.	23
Figure 17: History dashboard displaying recent AQI and pollutant trends for a selected city.	24
Figure 18: Model Insights dashboard presenting production-model metadata, evaluation results, explainability, and monitoring information.....	24
Figure 19: Successful GitHub Actions workflows providing continuous integration and scheduled machine-learning pipeline automation.	25
Figure 20: Live production-monitoring interface evaluating forecasts after their target timestamps have matured.	26
Figure 21 Successful production deployment of the Pearls AQI Predictor web application on Vercel.....	30
Figure 22: Final end-to-end architecture of the Pearls AQI Predictor, showing historical data processing, model development and management, serverless deployment, automated workflows, explainability, and production monitoring.	31

1. Executive Summary

Pearls AQI Predictor is an end-to-end machine learning system developed to forecast the Air Quality Index (AQI) for major cities in Pakistan over the next 24, 48, and 72 hours. The objective of the project was not limited to training a prediction model; it was to develop a complete machine learning workflow covering data acquisition, historical backfilling, exploratory data analysis, feature engineering, model experimentation, temporal validation, model management, automated pipelines, deployment, explainability, and production monitoring.

The original project brief proposed a serverless architecture in which weather and pollutant data would be collected from external APIs, engineered features would be stored in a feature store such as Hopsworks or Vertex AI, models would be trained and stored in a model registry, automated pipelines would run hourly and daily, and a web application would display real-time and forecasted AQI values. It also recommended experimenting with classical and deep-learning models and evaluating them using MAE, RMSE, and R^2 .

During implementation, several of the initially suggested cloud services introduced practical limitations. In particular, the Hopsworks environment repeatedly remained stuck while provisioning/allocating the required cluster, which prevented reliable progress with the hosted feature-store workflow. I therefore replaced the hosted feature-store dependency with a local Feast implementation using a SQLite-backed registry and online store. This retained the main feature-store concepts while allowing the complete pipeline to remain reproducible and free to operate.

The final system uses historical data from Open-Meteo and CAMS Global, live observations from OpenWeather, optional AQICN validation, 56 engineered model features, XGBoost production models, MLflow for model management, Feast for feature-contract validation, FastAPI for inference, React and TypeScript for the dashboard, GitHub Actions for automation, and Vercel for free deployment.

Three forecasting horizons are supported: 24h, 48h, and 72h. XGBoost was selected as the production model after temporal walk-forward evaluation against Persistence, Ridge Regression, Random Forest, and deep-learning candidates including CNN, GRU, and CNN-LSTM. The final untouched holdout set contained 50,544 observations for each forecasting horizon, and the resulting performance was:

Table 1: Final holdout performance of the selected XGBoost forecasting models.

Forecast Horizon	MAE	RMSE	R^2
24 Hours	14.91	21.16	0.791
48 Hours	19.35	26.89	0.654
72 Hours	20.47	28.25	0.613

The final system therefore represents a complete practical ML lifecycle rather than an isolated predictive notebook.

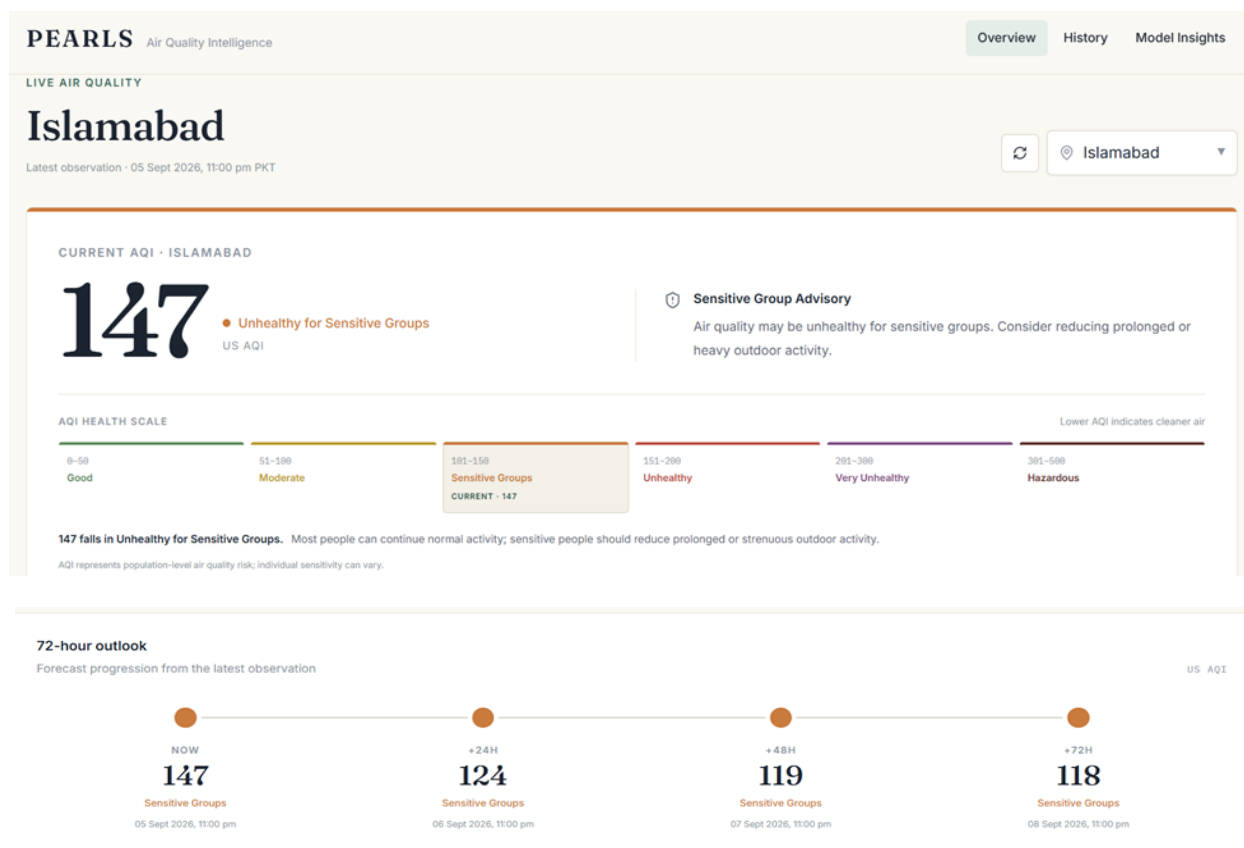


Figure 1 Pearls AQI Predictor production dashboard displaying current air quality and 24-, 48-, and 72-hour AQI forecasts.

2. Project Objective and Requirements

The main objective was to predict AQI up to three days into the future using a serverless and automated machine-learning architecture. The company project brief required the following major components:

- collection of weather and pollutant information from external APIs;
- historical backfilling of training data;
- feature and target engineering;
- use of a feature store;
- experimentation with classical and advanced forecasting models;
- a model registry;
- evaluation using RMSE, MAE, and R^2 ;

- hourly feature-generation automation;
- daily model-training automation;
- explainability using SHAP or LIME;
- hazardous AQI alerts;
- an interactive dashboard;
- documentation of the completed system.

Rather than implementing these as disconnected components, I designed the project as one integrated pipeline:

```

Historical / Live Environmental Data
↓
Data Validation
↓
AQI Calculation
↓
Feature Engineering
↓
Temporal ML Dataset
↓
Walk-Forward Validation
↓
Model Benchmarking
↓
XGBoost Model Selection
↓
MLflow Model Release
↓
FastAPI Prediction API
↓
React Dashboard
↓
Live Forecast Evaluation & Monitoring

```

3. Technology Selection and Free Alternatives

A significant part of the implementation involved selecting tools that could satisfy the project requirements while remaining usable under free-tier constraints. The original brief suggested Hopsworks or Vertex AI for the feature store and mentioned tools such as Airflow/GitHub Actions for scheduling, Streamlit/Gradio for presentation, and Flask/FastAPI for serving. I used the following alternatives:

Table 2 Comparison of originally suggested technologies and the free alternatives adopted during implementation.

Requirement	Suggested / Considered	Final Choice	Reason
Feature Store	Hopsworks / Vertex AI	Feast + local SQLite	Hopsworks cluster provisioning remained stuck; Feast was local, reproducible and free
Model Registry	Hosted registry/cloud registry	MLflow local registry + exported	Free and easy to reproduce

		deployment metadata	
Historical data	AQICN / OpenWeather examples	Open-Meteo + CAMS Global	Better historical coverage without paid historical API access
Live AQI/weather	AQICN / OpenWeather	OpenWeather + optional AQICN validation	Free tier available and suitable for current observations
Pipeline scheduler	Apache Airflow / GitHub Actions	GitHub Actions	No infrastructure server required
Backend	Flask / FastAPI	FastAPI	Typed APIs, automatic OpenAPI docs, cleaner ML serving
Dashboard	Streamlit / Gradio	React + TypeScript	More professional and flexible production UI
Deployment	Render / cloud platforms	Vercel	Free deployment without requiring paid infrastructure/card setup
Monitoring storage	Persistent cloud database	Git orphan branch + CSV ledger	Durable, free, auditable and compatible with serverless deployment

The Feast implementation used a local registry and SQLite online store, allowing the feature-store concepts to remain part of the architecture without depending on Hopsworks infrastructure. The resulting Feast configuration used a local provider with SQLite storage.

Hopsworks Issue

Hopsworks was initially considered because it aligned closely with the project brief. However, it repeatedly remained stuck during cluster allocation/provisioning, preventing access to a usable feature-store environment. Given the project deadline and need for a reproducible free solution, I moved to **Feast**, implementing a local feature repository and validating the production feature contract without relying on managed cloud infrastructure.

```
(.venv) PS F:\pearls-aqi-predictor\feature_repo\feature_repo> feast version
>> Get-Content .\feature_store.yaml
>> feast apply
F:\pearls-aqi-predictor\feature_repo\feature_repo\feature_definitions.py:20:
next release. Please specify a value_type for entity 'city'.
    city = Entity(
Applying changes for project pearls_aqi_predictor
Updated feature view aqi_features
    batch_source: type: BATCH_FILE
timestamp_field: "event_timestamp"
created_timestamp_column: "created"
file_options {
  uri: "data/model_features.parquet"
}
data_source_class_type: "feast.infra.offline_stores.file_source.FileSource"
name: "aqi_model_features_source"
meta {
  created_timestamp {
    seconds: 1788018292
    nanos: 890545000
  }
  last_updated_timestamp {
    seconds: 1788018292
    nanos: 890545000
  }
}
}

-> type: BATCH_FILE
timestamp_field: "event_timestamp"
created_timestamp_column: "created"
file_options {
  uri: "data/model_features.parquet"
}
data_source_class_type: "feast.infra.offline_stores.file_source.FileSource"
name: "aqi_model_features_source"
meta {
  created_timestamp {
    seconds: 1788674779
    nanos: 144168000
  }
  last_updated_timestamp {
    seconds: 1788674779
    nanos: 144168000
  }
}
}

No changes to infrastructure
```

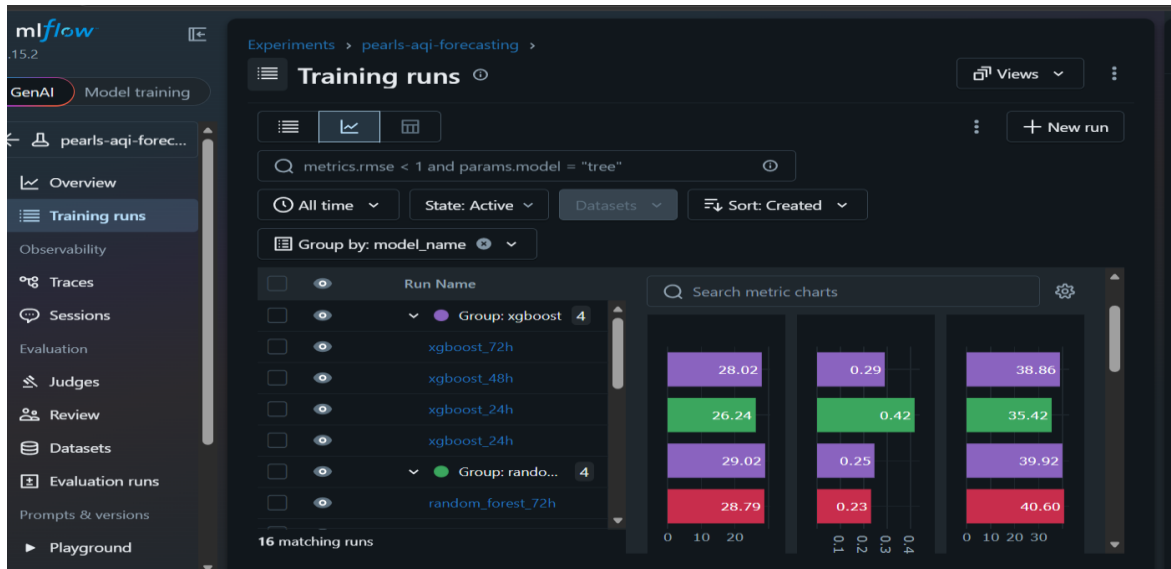


Figure 2 Feast feature store configuration and successful feature repository validation.

4. Data Collection and Historical Backfilling

The first major technical task was obtaining enough historical data to train forecasting models. The initial brief listed AQICN and OpenWeather as examples but explicitly allowed exploration of other data sources. Historical access through live-oriented APIs was either limited or unsuitable for the size of dataset required, so I selected **Open-Meteo historical weather data together with CAMS Global air-quality data**.

The historical backfill covered approximately: **August 2022 to August 2026**

The initial acquisition contained ten cities and generated a master dataset of 356,640 hourly rows across 17 raw columns. Each city had continuous hourly observations with no duplicate timestamps or missing observations after processing.

The original cities included:

- Faisalabad
- Islamabad
- Karachi
- Lahore
- Multan
- Peshawar
- Quetta
- Rahim Yar Khan
- Rawalpindi
- Sialkot

During dataset auditing, Rawalpindi was later excluded because its atmospheric grid values duplicated Islamabad's effective CAMS representation. Retaining both would have introduced unnecessary duplicated spatial information into the model. The final ML dataset therefore contains **nine cities**.

The backfill process downloaded weather and air-quality information year-by-year and city-by-city. For example, the pipeline retrieved data from 2022 through 2026 for Faisalabad and produced 35,664 continuous hourly records with zero missing values, zero duplicated timestamps, and no non-hourly gaps. The final ML-ready dataset contained:

- **319,473 observations**
- **9 cities**
- **56 production features**
- no missing feature cells;
- no duplicate entity/timestamp keys;
- Time range from August 2022 to August 2026.

```

PEARLS AQI - HISTORICAL BACKFILL SUMMARY
=====
Rows: 356,640
Columns: 17
Cities: 10
Start: 2022-08-04 00:00:00+00:00
End: 2026-08-28 23:00:00+00:00
Missing values: 0
Duplicate city/timestamps: 0

Rows per city:
city
Faisalabad      35664
Islamabad       35664
Karachi          35664
Lahore           35664
Multan           35664
Peshawar         35664
Quetta           35664
Rahim Yar Khan   35664
Rawalpindi       35664
Sialkot          35664

```

Figure 3 Historical environmental-data backfill summary prior to final ML dataset filtering.

Timestamp	City	Temp	Humidity	PM2.5	PM10	AQI	AQI +24h	AQI +48h	AQI +72h
2022-08-07 23:00	Faisalabad	26.6	93	44.5	66.8	130.0	153.0	147.0	151.0
2022-08-08 00:00	Faisalabad	27.3	92	48.1	71.5	127.0	153.0	149.0	148.0
2022-08-08 01:00	Faisalabad	27.6	90	48.4	73.6	124.0	153.0	151.0	144.0
2022-08-08 02:00	Faisalabad	28.4	87	47.3	73.1	122.0	154.0	152.0	140.0
2022-08-08 03:00	Faisalabad	29.7	82	45.5	71.5	121.0	154.0	153.0	137.0
2022-08-08 04:00	Faisalabad	31.3	74	44.1	69.6	120.0	154.0	153.0	136.0
2022-08-08 05:00	Faisalabad	32.4	69	43.9	69.2	120.0	154.0	153.0	136.0
2022-08-08 06:00	Faisalabad	33.2	65	41.6	65.0	120.0	154.0	153.0	136.0

Figure 4 Sample of the final ML-ready dataset containing meteorological variables, pollutant measurements, AQI values, and multi-horizon forecasting targets.

5. AQI Target Construction

The pollutant API data alone did not directly provide the exact forecasting target needed by the models, so AQI targets were calculated during preprocessing. The system uses pollutant concentrations, particularly PM2.5 and PM10, and maps them to AQI categories based on the implemented U.S. EPA particle-pollution breakpoint methodology. Instead of predicting a single next-hour value, I created three future target variables:

```
target_aqi_24h  
target_aqi_48h  
target_aqi_72h
```

These correspond to AQI values 24, 48, and 72 hours after a given feature timestamp. This transformation converted the problem into **multi-horizon supervised regression**, with a separate production model trained for each forecasting horizon.

6. Exploratory Data Analysis

EDA was performed before finalizing the feature and modeling pipeline, as requested in the project specification. The analysis focused on understanding:

- AQI distributions;
- city-level variations;
- pollutant behavior;
- seasonal and temporal changes;
- lag relationships;
- correlations between pollutant concentrations and AQI;
- weather variables and air quality;
- frequency of AQI categories;
- differences among forecasting horizons.

The analysis showed that AQI is not distributed uniformly across cities or time. Pollution patterns are strongly temporal, meaning recent AQI and pollutant values contain important information about near-future AQI.

PM2.5 emerged as one of the strongest operational indicators in many production explanations. For example, live model explanations frequently include the 24-hour PM2.5 rolling mean, current PM2.5, and previous AQI lag values among the largest model contributions.

This justified the inclusion of temporal lag and rolling-window features rather than relying solely on instantaneous weather conditions.

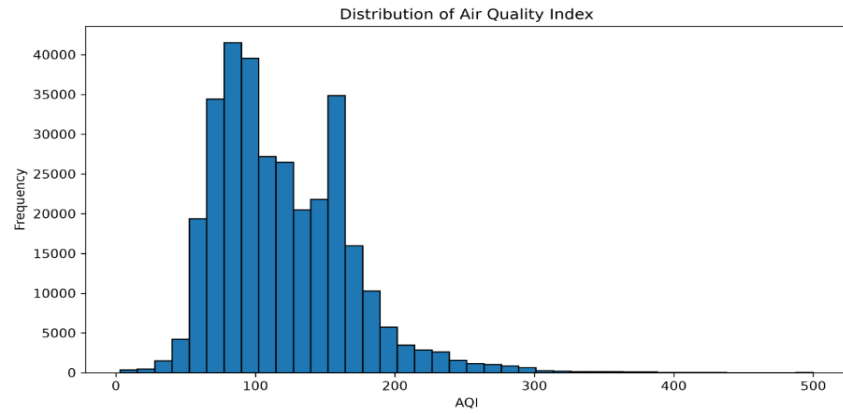


Figure 5: Exploratory analysis of AQI distribution and city-level average air quality across the final dataset.

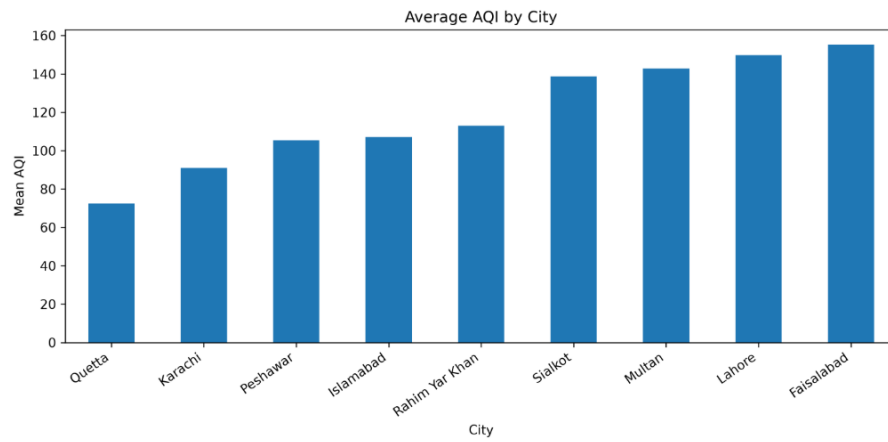


Figure 6: Average historical AQI across the nine Pakistani cities included in the final modeling dataset.

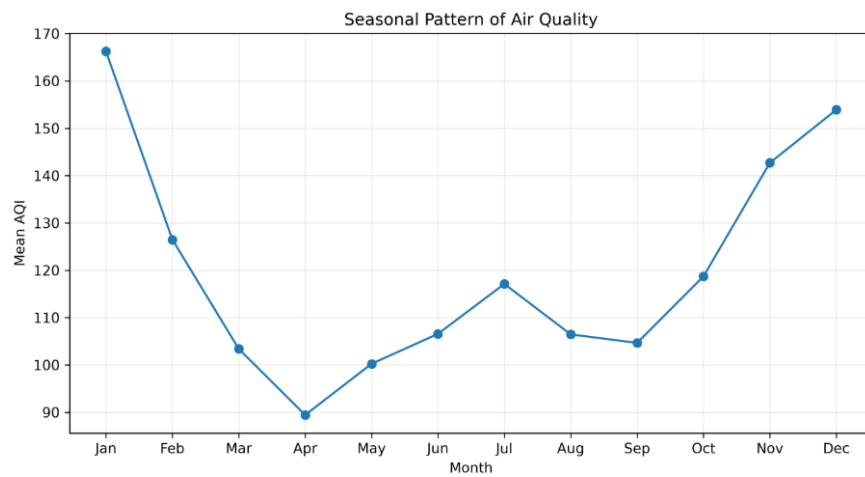


Figure 7: Monthly mean AQI illustrating seasonal variation in historical air-quality conditions.

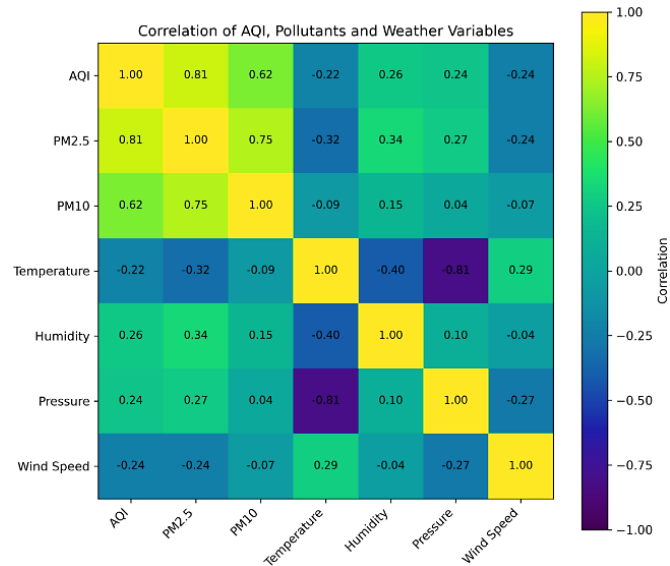


Figure 8 Correlation matrix showing relationships among AQI, particulate pollutants, and major meteorological variables.

The EDA confirmed considerable variation across cities and time periods and demonstrated strong temporal dependence in AQI values. This motivated the use of lag variables, rolling pollutant statistics, temporal features and city/location information in the final model feature set.

7. Feature Engineering

The initial project requirements specifically recommended time-based variables and derived AQI changes. I expanded this concept into a larger time-series feature set. The final production model receives **56 features**. The features can be grouped into:

Current Environmental Features

Current pollutant and meteorological conditions, including variables such as:

- PM2.5
- PM10
- temperature
- humidity
- atmospheric pressure
- wind information
- other pollutant concentrations

Temporal Features

Examples include:

- hour

- day
- month
- cyclical or seasonal information

AQI Lag Features

Examples include previous AQI observations such as:

- AQI lagged by recent hours
- AQI 24 hours previously
- AQI 48 hours previously
- AQI 72 hours previously

Rolling Statistics

Examples include:

- rolling AQI means
- PM2.5 rolling means
- pollutant rolling statistics
- rate/change indicators

Spatial Features

City/location information such as latitude and longitude was included so that one general model per horizon could learn geographic differences between locations. This engineering stage was important because future AQI depends heavily on recent pollution behavior and recent environmental context, not just the current raw observation.

```
=====
PEARLS AQI - FINAL ML DATASET
=====
Rows:          319,473
Dataset columns: 74
Cities:        9
Model features: 56
Start:         2022-08-07 23:00:00+00:00
End:           2026-08-25 23:00:00+00:00
Missing features: 0
Duplicate keys: 0
=====
```

Figure 9 Summary and quality validation of the final ML-ready dataset used for model development.

8. Feature Store Implementation

The feature-store stage underwent one of the most significant architectural changes in the project. As mentioned previously, Hopsworks was attempted first but became impractical because its managed cluster did not complete provisioning. Vertex AI would have introduced additional cloud configuration/billing complexity. I therefore implemented **Feast** locally. The validated Feast dataset was explicitly checked for:

- 319,473 source rows;
- nine cities;
- 56 features;
- no missing feature cells;
- no duplicate timestamp/entity keys;
- exclusion of Rawalpindi.

In the final serverless production deployment, I deliberately avoided forcing a stateful Feast service into every Vercel request. Instead, Feast serves as the validated **feature contract and development feature-store layer**, while the API builds the same exact 56-feature schema in memory for inference.

This approach provided the engineering benefits of consistent feature definitions while remaining compatible with a lightweight serverless deployment.

9. Train/Validation/Test Strategy

A normal random train-test split would be inappropriate for this project because this is time-series forecasting. Random splitting could allow later observations to influence earlier predictions and create overly optimistic performance estimates. I therefore used **temporal walk-forward validation**. Three development folds were created:

```
Fold 1
Past data → validation period 1
Fold 2
More past data → validation period 2
Fold 3
More past data → validation period 3
```

Each model was evaluated across all three folds. The **model-selection metric was mean validation MAE across the three walk-forward folds**. Worst-fold MAE and standard deviation were available as additional stability indicators. Only after the model family had been selected was the final untouched holdout evaluated. The final training and test configuration was:

Table 3 Final temporal training and holdout dataset configuration.

Item	Value
Training rows	268,281
Final holdout rows	50,544
Cities	9
Features	56
Training end	31 Dec 2025
Test start	4 Jan 2026
Test end	25 Aug 2026

The project artifacts explicitly record that model selection was frozen **before** final test evaluation. This was an important step in ensuring that the reported performance is credible rather than repeatedly tuning the model against the final test set.

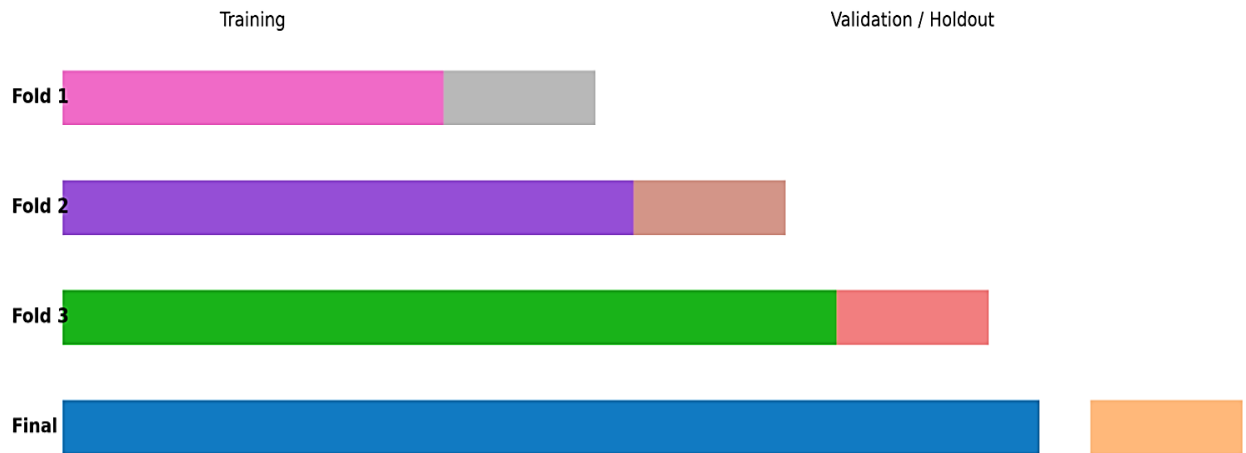


Figure 10 Temporal walk-forward validation strategy and isolated final holdout period used to prevent time-series data leakage.

10. Models Experimented With

The brief requested experimentation beyond a single algorithm, including classical and advanced models. Following candidates were evaluated:

Table 4 Forecasting models evaluated during model experimentation and benchmarking.

Model	Category	Purpose
Persistence	Baseline	Predicts using recent AQI behavior
Ridge Regression	Linear ML	Simple regularized baseline
Random Forest	Ensemble	Nonlinear tree-based model
XGBoost	Gradient boosting	High-performance tabular model
CNN	Deep Learning	Learns local patterns from sequences
GRU	Deep Learning	Recurrent time-series model

CNN-LSTM	Deep Learning	Combined spatial/temporal sequence modeling
----------	---------------	---

The deployed model metadata confirms that Persistence, Ridge, Random Forest, XGBoost, CNN, GRU, and CNN-LSTM were all evaluated, while XGBoost became the production algorithm for all three horizons.

Classical Model Walk-Forward Results

The strongest classical validation results were:

Table 5 Mean walk-forward validation MAE of the classical forecasting models across all prediction horizons.

Horizon	XGBoost MAE	Random Forest MAE	Ridge MAE	Persistence MAE
24h	13.3346	13.6623	13.6634	15.4005
48h	19.1634	19.3003	19.4529	20.7931
72h	20.9650	21.1477	21.2856	22.9612

XGBoost achieved the lowest mean validation MAE at every horizon. Deep-learning models were also trained and evaluated, including sequence-based CNN experiments whose validation outputs were stored independently for comparison. Ultimately, the more complex neural models did not provide a sufficiently reliable improvement over XGBoost to justify greater runtime and serving complexity.

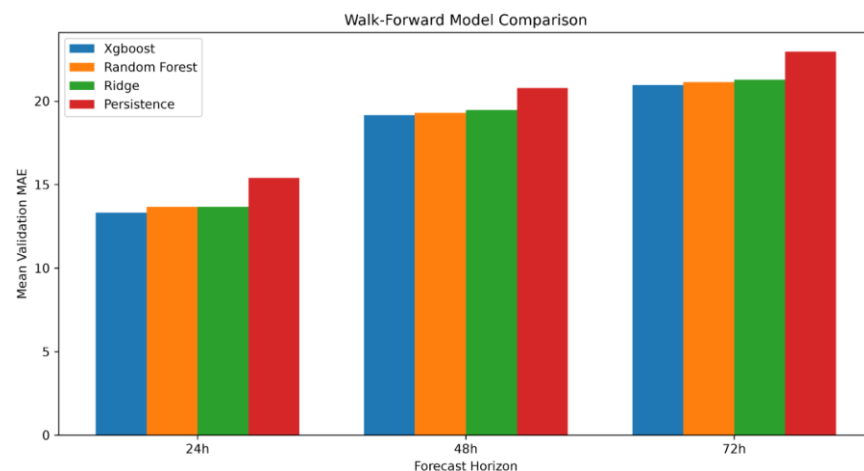


Figure 11: Comparison of candidate forecasting models using mean walk-forward validation MAE across the three forecast horizons.

11. Why XGBoost Was Selected

XGBoost became the production model for all horizons because it provided the strongest balance of:

- lowest mean walk-forward validation MAE;
- stable performance across time folds;
- fast training;
- very fast inference;
- strong nonlinear modeling capability;
- compatibility with tabular lag/rolling features;
- low deployment overhead;
- and interpretable feature contributions.

The mean walk-forward metrics for the selected XGBoost models were:

Table 6 Mean walk-forward validation performance of the selected XGBoost models.

Horizon	Mean MAE	Mean RMSE	Mean R ²
24h	13.3346	19.0184	0.8393
48h	19.1634	25.9224	0.7044
72h	20.9650	27.9551	0.6605

These are **development validation metrics**, not the final test metrics. That distinction is important because it prevents reporting validation results as if they were unseen production performance.

12. Final Holdout Evaluation

After model selection was frozen, each XGBoost model was retrained using the final training partition and evaluated once on the untouched holdout. The final results were:

Table 7 Final XGBoost holdout performance compared with the persistence baseline.

Horizon	MAE	RMSE	R ²	Persistence MAE	Improvement
24h	14.9095	21.1642	0.7908	17.9547	16.96%
48h	19.3461	26.8949	0.6543	22.5511	14.21%
72h	20.4679	28.2486	0.6128	24.3108	15.81%

These values came from 50,544 holdout observations for each horizon. The system also calculated AQI-category accuracy:

- **24h:** 70.51%
- **48h:** 61.86%
- **72h:** 60.05%

and error-tolerance statistics. For example, at the 24h horizon approximately 74.78% of predictions were within 20 AQI points and 88.27% were within 30 AQI points. These results show the expected forecasting behavior: performance decreases as the prediction horizon becomes longer because uncertainty accumulates over time.

Metric Definitions

- **MAE:** Measures the average absolute difference between predicted and actual AQI. Lower is better.
- **RMSE:** Penalizes larger prediction errors more heavily. Lower is better.
- **R²:** Measures how much of the target variation is explained by the model. Closer to 1 is better.

Final holdout evaluation

One-time final test results for the selected production models

HORIZON	MAE	RMSE	R ²	WITHIN ±30	AQI HEALTH BAND MATCH	BASLINE MAE	IMPROVEMENT
24h	14.91	21.16	0.791	88.3%	70.5%	17.95	+17.0%
48h	19.35	26.89	0.654	80.4%	61.9%	22.55	+14.2%
72h	20.47	28.25	0.613	77.8%	60.1%	24.31	+15.8%

Figure 12 Performance of the selected XGBoost models on the frozen final holdout dataset.

13. Explainability

The project requirement suggested SHAP or LIME for explaining feature importance. For the final XGBoost models, I used native XGBoost prediction contributions, which are TreeSHAP-style additive feature contributions. Each prediction therefore contains:

- a model base value;
- the most influential features;
- each feature's observed value;
- the size of its contribution;
- whether it increased or decreased the prediction.

For example, production explanations have identified features such as:

- PM2.5 rolling mean over 24 hours;
- AQI 72 hours ago;
- current PM2.5;
- rolling AQI mean;
- and geographic longitude

as meaningful contributors to particular predictions. These explanations describe the model's behavior and are not presented as proof of causal environmental effects.

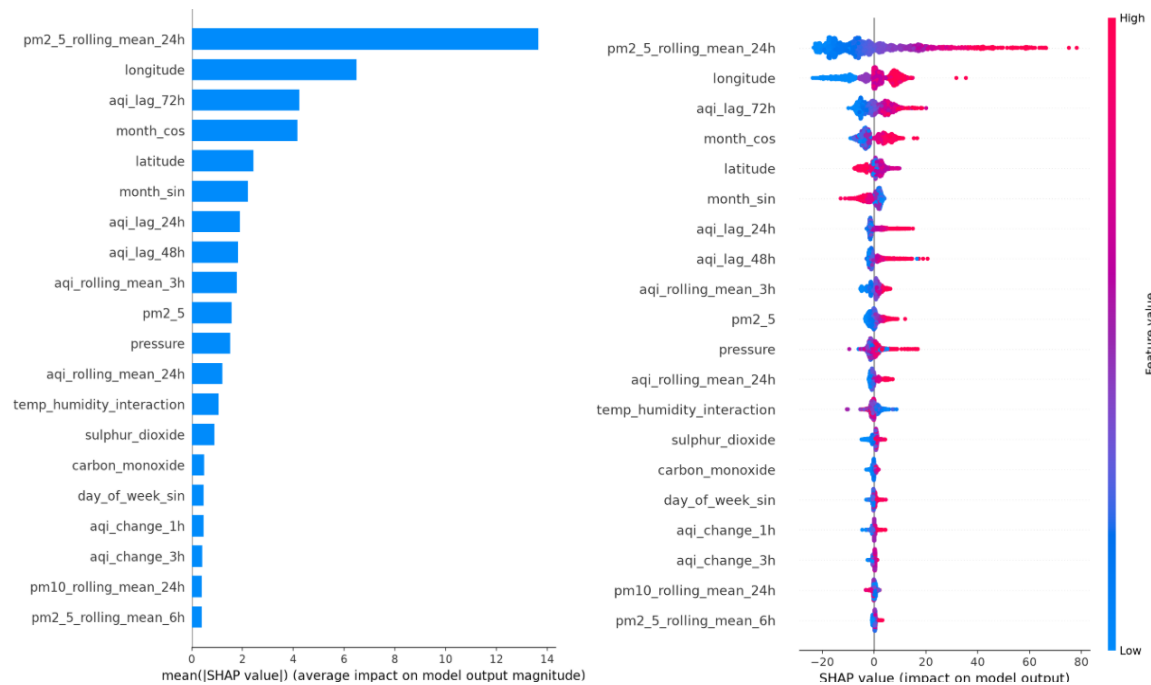


Figure 13: XGBoost feature-contribution explanation showing the variables with the strongest influence on an individual AQI forecast.

14. Model Registry and MLOps

Model management was implemented through **MLflow**. Each production horizon has a corresponding champion alias:

24h → champion-24h
 48h → champion-48h
 72h → champion-72h

The deployed model is registered as `pearls-aqi-xgboost`. The API itself exposes these registry aliases for transparency.

For production deployment, validated MLflow information is exported as deployment metadata along with the actual model artifacts. This avoids depending on a permanently running MLflow server while still retaining MLflow as the model-release source of truth. The MLOps design therefore separates:

Development / Model Management

MLflow + Feast

↓

Validated release artifacts

↓

Serverless production inference

This made the final solution considerably easier to deploy under free serverless constraints.

Version	Registered at	Created by	Tags	Aliases	Description
Version 3	08/29/2026, 08:16:44 PM		forecast_horizon: 72h model_type: xgboost deployment_status: champion selection_frozen_before_test training_data_end: 2025-12-: final_test_year: 2026	@ champion-72h	
Version 2	08/29/2026, 08:16:36 PM		forecast_horizon: 48h model_type: xgboost deployment_status: champion selection_frozen_before_test training_data_end: 2025-12-: final_test_year: 2026	@ champion-48h	
Version 1	08/29/2026, 08:16:26 PM		forecast_horizon: 24h model_type: xgboost deployment_status: champion selection_frozen_before_test training_data_end: 2025-12-: final_test_year: 2026	@ champion-24h	

Figure 14: MLflow model-management interface used to track and release the production XGBoost forecasting models.

15. Backend Development

The prediction API was implemented using **FastAPI**. The major endpoints include:

```
/api/v1/cities
/api/v1/forecast/{city}
/api/v1/history/{city}
/api/v1/models
/api/v1/performance/{city}
```

The backend performs:

- live environmental-data acquisition;
- recent-history bootstrap;
- feature engineering;
- model loading;
- inference;
- AQI categorization;
- hazardous-level alerts;
- explanation generation;
- model metadata retrieval;
- live performance calculation.

FastAPI was chosen instead of a purely dashboard-based serving approach because it cleanly separates the ML/backend layer from the user interface and automatically provides Swagger/OpenAPI documentation.

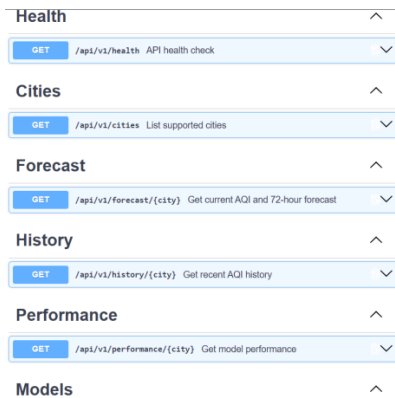


Figure 15: Swagger/OpenAPI documentation for the deployed FastAPI prediction service.

16. Frontend Dashboard

Although the brief suggested Streamlit or Gradio as possible frontend technologies, I chose **React + TypeScript + Vite** to create a more structured production dashboard. The dashboard consists primarily of:

Overview displays:

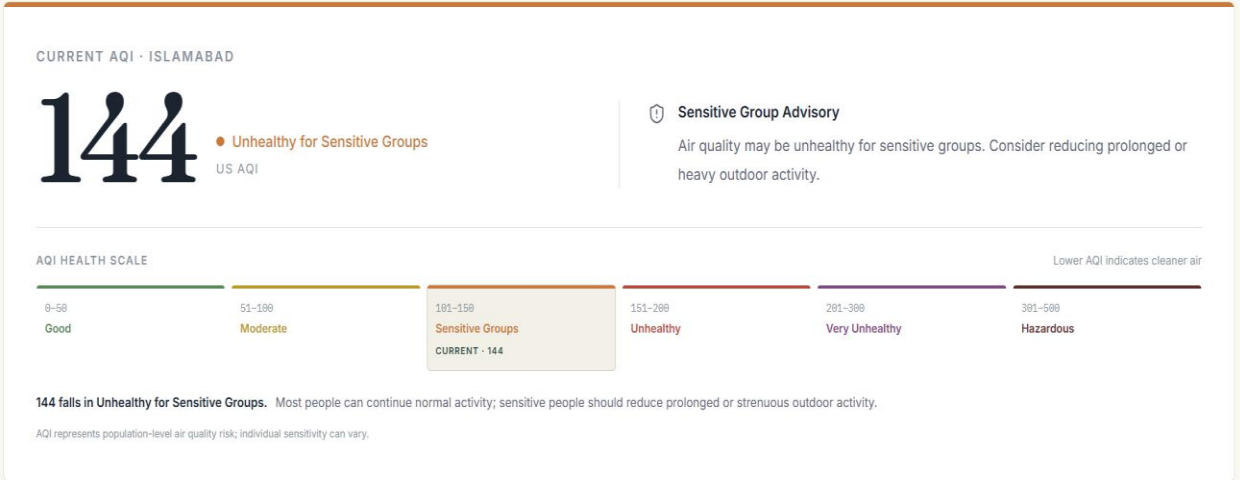
- selected city;
- current AQI;
- current environmental conditions;
- AQI category;
- 24/48/72-hour predictions;
- forecast alerts;
- explanation information;
- model reliability information.

History displays recent:

- AQI values;
- pollutant history;
- trend charts;
- descriptive statistics;
- and selectable history ranges.

Model Insights displays:

- production model information;
- frozen holdout metrics;
- explainability;
- and live matured-forecast performance.



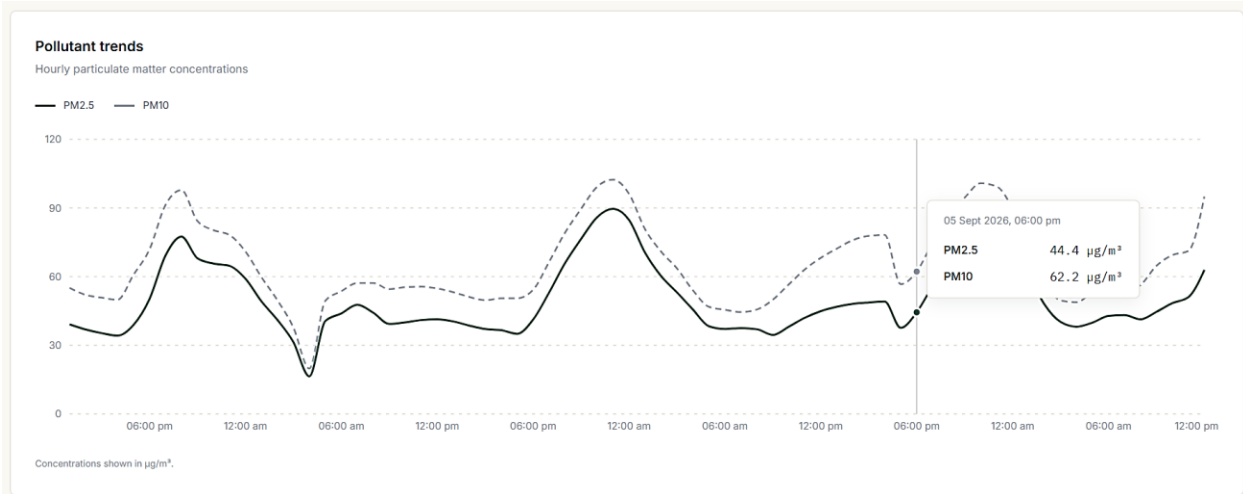


Figure 17: History dashboard displaying recent AQI and pollutant trends for a selected city.

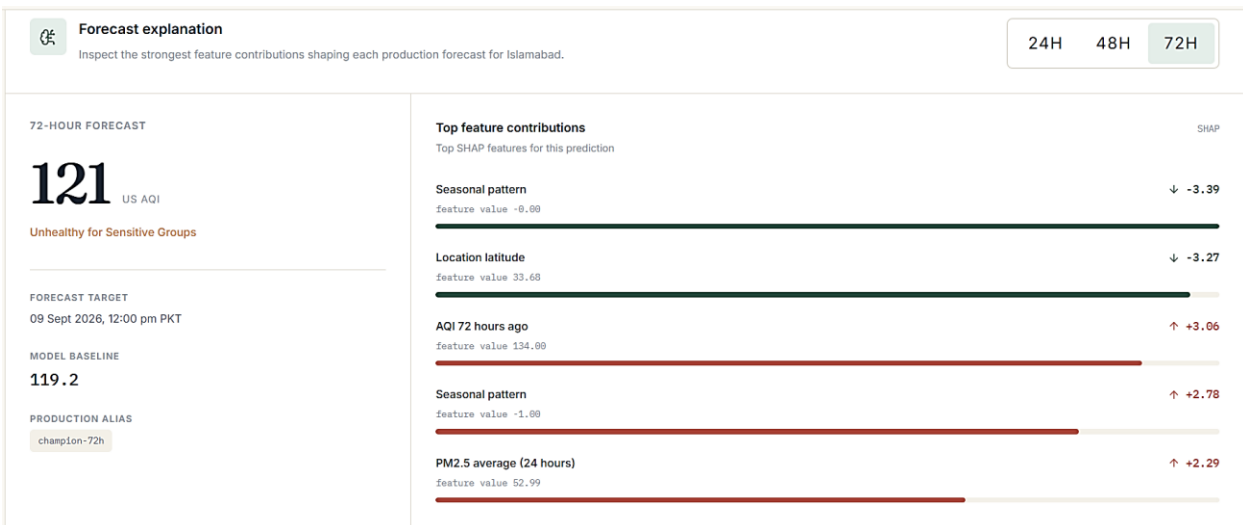
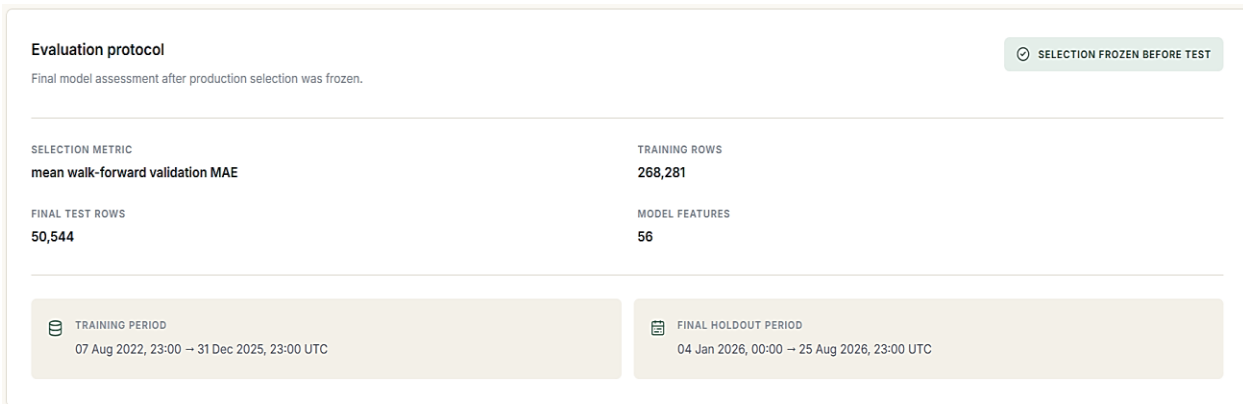


Figure 18: Model Insights dashboard presenting production-model metadata, evaluation results, explainability, and monitoring information.

17. Automation with GitHub Actions

The project brief required the feature pipeline to run hourly and the training pipeline to run daily. I implemented the automation using **GitHub Actions**, which was selected instead of Apache Airflow because it requires no separately hosted scheduler. The automation includes:

Continuous Integration

Validates:

- backend linting;
- automated backend tests;
- frontend linting;
- frontend production build.

Hourly Production Pipeline

The hourly job:

1. obtains the persistent monitoring ledger;
2. collects current/recent input data;
3. evaluates forecasts that have reached their target time;
4. generates new 24h/48h/72h forecasts;
5. updates the ledger;
6. commits the updated ledger to a dedicated branch.

Daily Development Workflow

The daily workflow trains/evaluates development candidates against protected validation data. Importantly, this workflow does **not** use the final holdout for automatic model reselection.

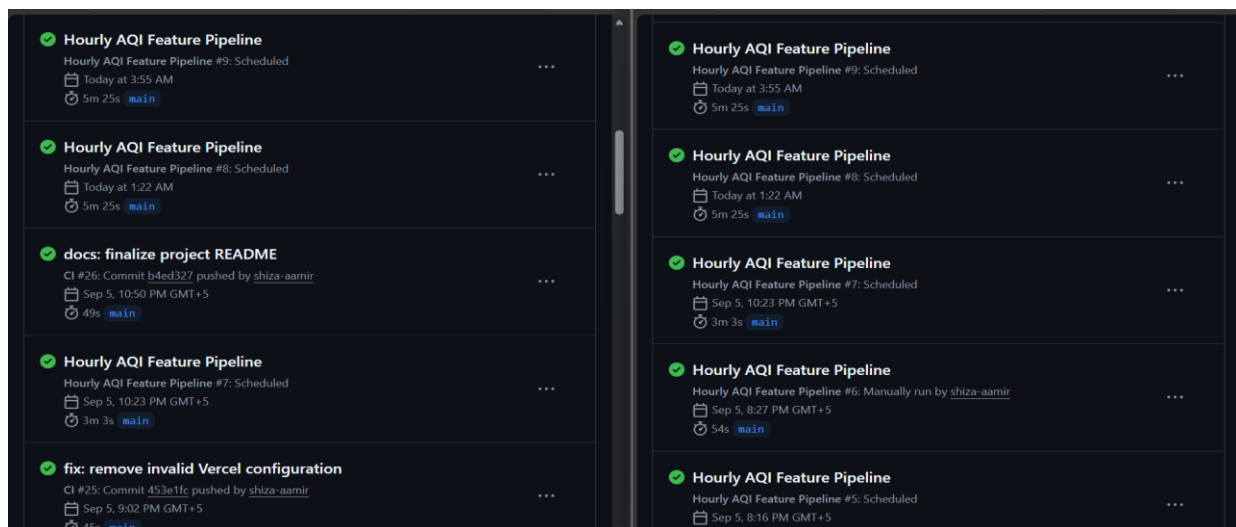


Figure 19: Successful GitHub Actions workflows providing continuous integration and scheduled machine-learning pipeline automation.

18. Production Monitoring

Future AQI forecasts cannot be evaluated until their target timestamps are reached. For example, a forecast generated on **5 September at 10:00** becomes evaluable after **24, 48, or 72 hours**, depending on its forecast horizon.

To support this, the system maintains a **forecast ledger** containing the city, forecast creation time, target timestamp, horizon, predicted AQI and category, actual AQI after maturity, absolute error, category correctness, and evaluation timestamp.

The dashboard distinguishes between two types of performance:

- **Frozen Holdout Evaluation:** Formal model performance measured on a large, fixed unseen test dataset.
- **Live Monitoring:** Ongoing evaluation of real production forecasts as their target timestamps mature.

This separation ensures that small early production samples are not misrepresented as overall model accuracy.

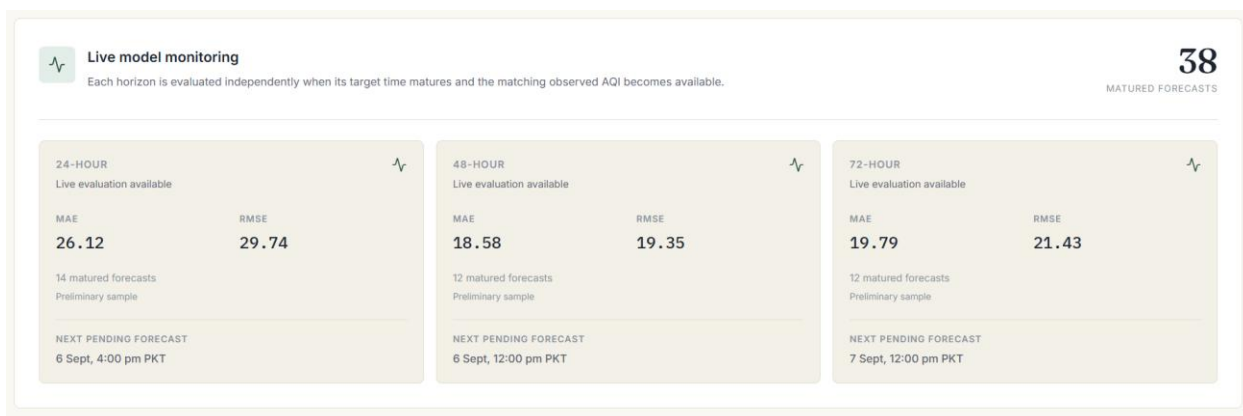


Figure 20: Live production-monitoring interface evaluating forecasts after their target timestamps have matured.

19. Serverless Persistence Challenge

One of the largest production issues emerged after deploying the backend. Vercel serverless functions have ephemeral local storage. Initially, forecast-monitoring data written locally could disappear after the function instance restarted. This caused the live monitoring ledger to reset, even though the model itself was working correctly.

Solution

I redesigned the monitoring architecture so that the hourly GitHub Action maintains a durable CSV ledger in a separate Git branch named: `monitoring-data`

The branch contains only the monitoring ledger and related metadata, while the application code remains on main. The workflow became:

```
GitHub Action
↓
Read monitoring-data ledger
↓
Evaluate matured forecasts
↓
Generate new forecasts
↓
Update CSV
↓
Push monitoring-data branch
↓
Vercel API reads durable raw CSV
```

This created a completely free persistence layer without requiring a paid database. This also illustrates one of the strongest lessons from the project: the ML model is only one small part of a production ML application. Data persistence, scheduling and deployment constraints can require as much engineering as model training.

20. Other Problems Faced and Their Resolution

Several problems appeared during the project lifecycle.

20.1 Historical API Limitations

Problem: The example APIs in the brief were suitable for live data but did not provide the required historical coverage conveniently under free constraints.

Resolution: Used Open-Meteo historical services and CAMS Global data for long-term training data, while retaining OpenWeather for live observations.

20.2 Hopsworks Cluster Provisioning

Problem: The Hopsworks environment repeatedly remained stuck while allocating/provisioning the managed cluster.

Resolution: Replaced the hosted feature store with Feast using a local feature repository and SQLite storage.

20.3 Large Dataset Size

Problem: The ML-ready CSV became very large; one generated version was over 225 MB.

Resolution: Used Parquet for efficient internal storage and excluded generated/raw data from unnecessary production deployment.

20.4 Time-Series Leakage Risk

Problem: A random train/test split would create temporal leakage.

Resolution: Implemented walk-forward validation and froze model selection before final holdout evaluation.

20.5 Deep Models Were More Expensive Without Clear Benefit

Problem: CNN/GRU/CNN-LSTM required more training/runtime complexity.

Resolution: Compared them fairly against classical models and selected XGBoost based on validation performance rather than model complexity.

20.6 GitHub Action Parquet Error

At one point, the production workflow failed because the runner did not contain a compatible Parquet engine:

```
ImportError: Unable to find a usable engine;  
tried using: 'pyarrow', 'fastparquet'
```

The workflow log explicitly showed the missing `pyarrow` / `fastparquet` dependency.

Resolution: Installed `pyarrow` in the workflow environment so Pandas could load/save Parquet data correctly.

20.7 External API Timeout

Open-Meteo occasionally timed out during automated historical/recent-history retrieval. A production action recorded a 30-second read timeout.

Resolution: The live-history pipeline was designed with bootstrap and gap-recovery logic so that temporary API availability did not require reconstruction of the entire application.

20.8 History Endpoint Dependency

Problem: Initially, opening History could fail on a new serverless instance unless Forecast had already initialized local history.

Resolution: Changed the History service so it independently ensures/bootstrap its required recent history before responding.

20.9 Vercel Ephemeral Storage

Problem: Monitoring information written to local `/tmp` was not durable.

Resolution: Moved forecast-ledger persistence to the `monitoring-data` Git branch.

20.10 Free Hosting Constraints

Problem: Several deployment alternatives either required billing configuration, a payment card, or infrastructure incompatible with the free project constraint.

Resolution: Selected Vercel for both frontend and FastAPI backend and adapted the architecture for serverless execution rather than purchasing persistent cloud compute.

20.11 Vercel SPA Configuration Failure

Problem: While attempting to support direct client-side routes such as `/history`, a `vercel.json` configuration caused the production deployment to fail with: 'Invalid `vercel.json` file provided'

Resolution: Prioritized production stability, removed the problematic deployment configuration and restored the previously functioning production deployment instead of risking further architecture changes immediately before submission. This issue reinforced the importance of controlled deployment changes and rollback capability.

21. Testing and Validation

Testing was performed at several levels.

Backend

```
(.venv) PS F:\pearls-qi-predictor> ruff check src api pipelines tests
>> pytest
All checks passed!
tests\unit\ml\test_splitter.py .... [100%]

===== 71 passed in 16.84s =====
```

Frontend

```
(.venv) PS F:\pearls-qi-predictor\frontend> npm run lint
>> npm run build

dist/assets/HistoryPage-C9CcIk2T.js 375.82 kB | gzip: 107.31 kB

✓ built in 7.83s
[PLUGIN TIMINGS] Your build spent 36% of 7.8s inside plugin hooks (2.8s).
```

Production

Manual validation included:

- several supported cities;
- current AQI;
- all forecast horizons;
- historical views;
- model insights;
- explainability;
- live monitoring;
- FastAPI endpoints;
- Production deployment.

The project was ultimately deployed successfully after final QA.

22. Final Deployment

The final project is deployed using Vercel.

- **Frontend:** <https://pearls-aqi-predictor-two.vercel.app/>
- **Backend:** <https://pearls-aqi-predictor-api.vercel.app>
- **API Documentation:** <https://pearls-aqi-predictor-api.vercel.app/docs>
- **Source Code:** <https://github.com/shiza-aamir/pearls-aqi-predictor>

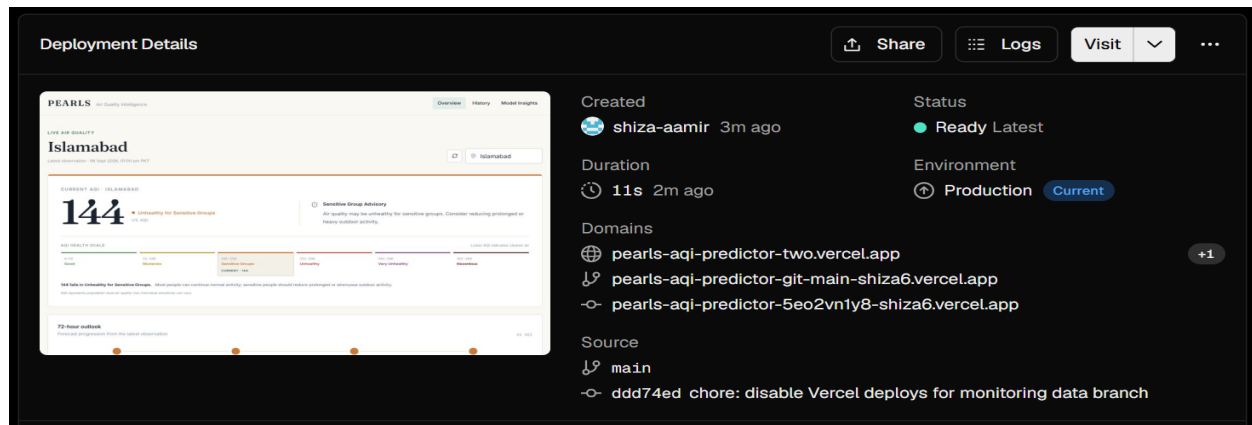


Figure 21 Successful production deployment of the Pearls AQI Predictor web application on Vercel.

23. Final System Architecture

The final architecture integrates the complete machine-learning lifecycle, from historical data acquisition and feature engineering to model development, deployment, automated forecasting, and production monitoring. Historical development and live production operate as connected

workflows, while GitHub Actions provides scheduled automation and the monitoring ledger preserves forecast results for subsequent evaluation.

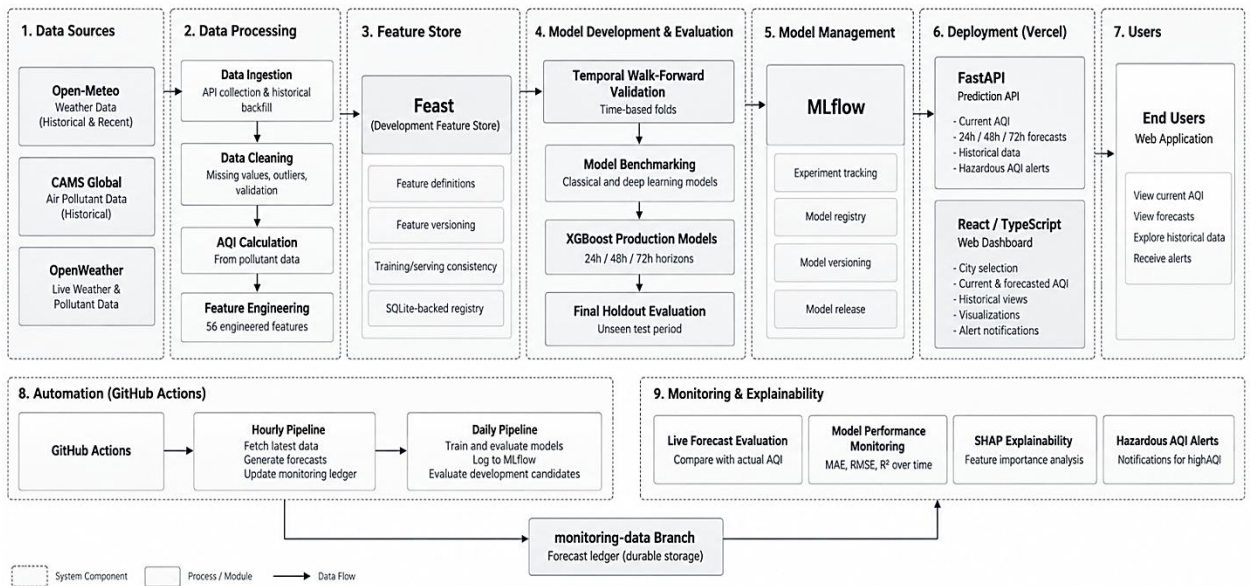


Figure 22: Final end-to-end architecture of the Pearls AQI Predictor, showing historical data processing, model development and management, serverless deployment, automated workflows, explainability, and production monitoring.

24. Results and Achievements

By the end of the project, the work had progressed beyond a standalone AQI regression experiment into a complete end-to-end machine learning system. The major components delivered are summarized below.

Table 8 Summary of the completed Pearls AQI Predictor system

Project Area	Completed Implementation
Data Engineering	Historical environmental-data ingestion, multi-city data cleaning, AQI target construction, and exploratory data analysis
Feature Engineering	56 production features with a Feast-validated feature-store workflow
Model Development	Classical and deep-learning experimentation with temporal walk-forward validation
Final Evaluation	Protected final holdout evaluation and comparison against the persistence baseline
Production Models	XGBoost forecasting models for 24-, 48-, and 72-hour AQI prediction
MLOps & Explainability	MLflow-managed model releases and feature-contribution-based prediction explanations
Backend Services	FastAPI prediction API with forecast, history, model, performance, and alert functionality
Frontend Application	React/TypeScript dashboard with current AQI, forecasts, historical views, model insights, and hazardous-AQI alerts

Automation & CI	Continuous integration, hourly forecast automation, and daily model-development automation using GitHub Actions
Production Monitoring	Durable forecast ledger with live evaluation of matured 24h, 48h, and 72h predictions
Deployment	Free serverless deployment of the frontend and backend on Vercel

The completed implementation therefore satisfies the central requirement of the project brief: an **end-to-end AQI prediction system with an automated, scalable pipeline and interactive dashboard**, covering the complete workflow from historical data acquisition and model development to deployment, automation, explainability, and production monitoring.

25. Limitations

Despite the successful implementation, several limitations remain.

- The historical air-quality component primarily relies on gridded CAMS atmospheric information rather than a dense network of ground-level monitoring stations. Therefore, the model should be interpreted as a forecasting prototype rather than a regulatory AQI authority.
- Forecast uncertainty increases with horizon, which is visible in the lower R^2 and higher error for the 48h and 72h models.
- The live production-monitoring dataset is still relatively small compared with the frozen holdout dataset and therefore should not yet replace holdout results as the primary performance benchmark.
- The serverless architecture also requires engineering compromises: stateful services such as a permanently running Feast or MLflow server are not hosted within the production request path.

26. Future Improvements

Future work could include:

- incorporating additional verified ground-monitor AQI sources;
- expanding coverage to more Pakistani cities;
- adding forecast uncertainty intervals;
- using satellite and wildfire/dust indicators;
- introducing drift detection;
- testing additional transformer-based time-series models;
- creating a managed persistent feature store when cloud resources permit;
- implementing automatic model promotion with strict governance;
- collecting several months of matured production forecasts for stronger real-world evaluation.

27. Conclusion

The Pearls AQI Predictor project provided practical experience across the complete machine-learning lifecycle. I began with a high-level project specification and transformed it into a functional system involving data engineering; feature engineering, forecasting, MLOps, API development, frontend development, CI/CD, deployment and monitoring.

A major learning outcome was that building a reliable ML system requires much more than selecting a high-performing algorithm. Challenges involving historical data access, feature-store infrastructure, time-series leakage, cloud free tiers, package dependencies, state persistence and deployment configuration were equally important.

Instead of becoming blocked when the originally suggested technologies were unavailable or impractical for example, the Hopsworks cluster-provisioning issue so I adapted the architecture using free and reproducible alternatives such as Feast, MLflow, GitHub Actions, Vercel and Git-based monitoring persistence.

The final XGBoost system achieved strong holdout results, particularly at the 24-hour horizon, while retaining meaningful performance at 48 and 72 hours. More importantly, the project was completed as an operational, automated, explainable and deployed ML application rather than simply a model-development exercise.